

课程笔记

KAIST CS492D Lecture 3: Denoising Diffusion Probabilistic Models (Part 2)

基于 Minhyuk Sung 授课内容整理

2025 年秋季

CS492C: Diffusion and Flow Models

Denoising Diffusion Probabilistic Models 2

LECTURE 4
MINHYUK SUNG

Fall 2025
KAIST

CS492(C): Diffusion and Flow Models (Fall 2025)

视频作者/频道: Minhyuk Sung (KAIST)

发布日期: 2025

视频时长: 59 分钟

视频链接: https://www.youtube.com/watch?v=5_t3uSvdrWg

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 回顾与动机	3
1.1 从 VAE 到扩散模型的动机	3
1.2 扩散模型的基本设定	3
1.3 本章小结	4
2 前向过程详解	4
2.1 前向转移分布的定义	4
2.2 前向跳跃分布 (Forward Jump Distribution)	4
2.3 本章小结	5
3 ELBO 分解与训练目标	5
3.1 从对数似然到 ELBO	5
3.2 ELBO 的逐项分解	5
3.3 真实后验转移分布的推导	6
3.4 本章小结	6
4 反向过程的参数化	6
4.1 反向转移分布的建模	6
4.2 简化后的训练目标	7
4.3 本章小结	7
5 三种等价的预测目标	7
5.1 方式一：均值预测 (Mean Prediction)	7
5.2 方式二：干净样本预测 (Clean Sample Prediction / x_0 -Prediction)	8
5.3 方式三：噪声预测 (Noise Prediction / ϵ -Prediction)	8
5.4 三种参数化的等价性与选择	9
5.5 本章小结	10
6 DDPM 训练算法	10
6.1 完整训练流程	10
6.2 训练的伪代码实现	10
6.3 关于权重项的讨论	11
6.4 本章小结	11
7 DDPM 采样算法	11
7.1 反向过程采样	11
7.2 采样的伪代码	12
7.3 采样过程中的 $\hat{x}_0$ 可视化	12
7.4 本章小结	13
8 扩散模型为何能建模复杂分布	13
8.1 单步高斯 vs. 多步高斯	13
8.2 本章小结	13

9 噪声预测与 Score Matching 的联系	14
9.1 从噪声预测到 Score Function	14
9.2 直觉理解	14
9.3 本章小结	14
10 总结与延伸	14
10.1 核心要点回顾	14
10.2 后续课程预告	15
10.3 拓展阅读	15

1 回顾与动机

本讲是 DDPM (Denoising Diffusion Probabilistic Models) 系列的第二部分。在上一讲中，我们了解了扩散模型的基本框架：从 VAE 的局限性出发，引入多层潜变量的思路，进而定义了扩散模型的前向过程和反向过程。本讲将从训练目标的推导开始，逐步推导出简化的训练损失函数，并引入三种等价的参数化方式：均值预测、干净样本预测和噪声预测。

1.1 从 VAE 到扩散模型的动机

回顾生成模型的核心目标：建模边际数据分布 $p(x_0)$ 。在 VAE 框架中，这通过先验分布 $p(z)$ (标准高斯) 和似然分布 $p_\theta(x|z)$ (解码器) 的组合来实现：

$$p_\theta(x_0) = \int p(z) p_\theta(x_0|z) dz$$

VAE 的局限在于，先验和似然都被建模为高斯分布，这限制了边际分布 $p_\theta(x_0)$ 的表达能力——它本质上是高斯分布的混合，可能无法充分捕捉复杂数据分布的多模态特性。

扩散模型的核心思想

扩散模型通过引入**多步**潜变量序列 $x_1, x_2, \dots, x_T$ 来增强似然分布的表达能力。虽然每一步的转移分布仍然是高斯分布，但通过多步非线性变换的组合，整体的似然分布可以表达远比单个高斯更复杂的分布。

1.2 扩散模型的基本设定

扩散模型的核心组件包括：

- **数据点** x_0 ：从真实数据分布 $q(x_0)$ 中采样的干净样本
- **最终潜变量** x_T ：标准高斯分布 $\mathcal{N}(0, I)$ 的样本
- **中间潜变量** $x_1, \dots, x_{T-1}$ ：介于数据和纯噪声之间的中间状态
- **前向过程** (Forward Process)：从 x_0 到 x_T ，逐步注入噪声
- **反向过程** (Reverse Process)：从 x_T 到 x_0 ，逐步去噪

与 VAE 的关键区别

在 VAE 中，编码器（后验分布）和解码器（似然分布）都需要学习。而在扩散模型中，前向过程是**预定义的**（不需要学习），我们只需要学习反向过程。此外，所有潜变量 x_t 与数据 x_0 具有**相同的维度**，这与 VAE 中潜变量通常维度更低不同。

关于马尔可夫性质：前向过程和反向过程都被定义为马尔可夫过程，即 x_t 仅依赖于 x_{t-1} （前向）或 x_{t+1} （反向），不依赖于更早的状态。形式上：

$$q(x_t|x_{t-1}, x_0) = q(x_t|x_{t-1})$$

这一性质大大简化了后续的数学推导。

1.3 本章小结

扩散模型通过预定义的前向噪声注入过程和可学习的反向去噪过程，在保持训练简单性的同时获得了强大的表达能力。与 VAE 不同，扩散模型不需要学习编码器，且所有潜变量与数据同维。

2 前向过程详解

2.1 前向转移分布的定义

前向过程的每一步转移被定义为一个特定形式的高斯分布：

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

其中各符号的含义如下：

- $\beta_t \in (0, 1)$: 第 t 步的噪声调度参数 (noise schedule)，是关于时间 t 的**非递减**函数
- $\sqrt{1 - \beta_t} x_{t-1}$: 均值——将上一步的值向原点方向缩放
- $\beta_t I$: 方差——注入各向同性的高斯噪声

前向过程的直观理解

每一步前向转移做了两件事：(1) 将当前值 x_{t-1} 乘以一个略小于 1 的系数 $\sqrt{1 - \beta_t}$ ，使其向原点收缩；(2) 添加方差为 β_t 的高斯噪声。当 β_t 接近 0 时， $x_t \approx x_{t-1}$ (几乎不变)；当 β_t 接近 1 时， x_t 几乎变为纯噪声。随着 t 增大， β_t 逐渐增大，噪声逐步积累，最终 x_T 趋近于标准高斯分布。

2.2 前向跳跃分布 (Forward Jump Distribution)

前向过程的一个关键优势是：我们不需要逐步执行前向过程，可以直接从 x_0 采样任意时间步 t 的 x_t 。定义 $\alpha_t = 1 - \beta_t$ 和 $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$ ，则有：

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I)$$

各符号说明：

- $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$: 累积保留系数，随 t 递减
- $\sqrt{\bar{\alpha}_t} x_0$: 均值——原始数据的缩放版本
- $(1 - \bar{\alpha}_t)I$: 方差——累积噪声的总量

前向跳跃分布的重要性

前向跳跃分布 $q(x_t|x_0)$ 允许我们在训练时**直接**从 x_0 采样任意时间步的 x_t ，无需逐步执行前向过程。这是 DDPM 训练高效性的关键所在。利用重参数化技巧：

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

这个公式建立了 x_0 、 x_t 和噪声 ϵ 之间的精确关系，后面推导噪声预测目标时会反复用到。

2.3 本章小结

前向过程通过预定义的高斯转移分布逐步向数据注入噪声。由于高斯分布的叠加仍为高斯，我们可以推导出闭式的前向跳跃分布 $q(x_t|x_0)$ ，从而在训练时直接从 x_0 一步采样 x_t ，无需迭代。

3 ELBO 分解与训练目标

3.1 从对数似然到 ELBO

与 VAE 类似，扩散模型的训练目标是最大化数据的对数似然 $\log p_\theta(x_0)$ 。由于直接优化不可行，我们转而优化其下界——证据下界 (ELBO)：

$$\log p_\theta(x_0) \geq \text{ELBO} = \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)} \right]$$

等价地，训练时我们最小化负 ELBO：

$$L = -\text{ELBO}$$

ELBO 与 VAE 的联系

ELBO 的推导与 VAE 完全一致：通过引入变分后验 $q(x_{1:T}|x_0)$ 并利用 Jensen 不等式得到下界。不同的是，在扩散模型中， q 是预定义的前向过程（不是学习得到的），因此 ELBO 的间隙 (gap) 由前向过程的设计决定。

3.2 ELBO 的逐项分解

通过利用马尔可夫性质和贝叶斯规则，ELBO 可以分解为 $T+1$ 个项的和。具体地，负 ELBO 可写为：

$$L = \underbrace{D_{\text{KL}}(q(x_T|x_0) \| p(x_T))}_{L_T(\text{先验匹配项})} + \sum_{t=2}^T \underbrace{D_{\text{KL}}(q(x_{t-1}|x_t, x_0) \| p_\theta(x_{t-1}|x_t))}_{L_{t-1}(\text{去噪匹配项})} + \underbrace{(-\log p_\theta(x_0|x_1))}_{L_0(\text{重建项})}$$

各项的含义：

- L_T (先验匹配项)：前向过程的终点分布 $q(x_T|x_0)$ 与先验 $p(x_T) = \mathcal{N}(0, I)$ 的 KL 散度。由于 q 是预定义的且当 T 足够大时 $q(x_T|x_0) \approx \mathcal{N}(0, I)$ ，此项近似为 0，**无需优化**。
- L_{t-1} (去噪匹配项)：这是核心项，要求学习的反向转移 $p_\theta(x_{t-1}|x_t)$ 匹配真实的后验转移 $q(x_{t-1}|x_t, x_0)$ 。
- L_0 (重建项)：最后一步从 x_1 重建 x_0 的负对数似然。

训练的核心：去噪匹配项 L_{t-1}

训练目标的关键是最小化去噪匹配项 L_{t-1} ，即使反向转移分布 $p_\theta(x_{t-1}|x_t)$ 尽可能接近真实后验转移 $q(x_{t-1}|x_t, x_0)$ 。这要求我们首先推导出 $q(x_{t-1}|x_t, x_0)$ 的闭式表达。

3.3 真实后验转移分布的推导

利用贝叶斯规则：

$$q(x_{t-1}|x_t, x_0) = \frac{q(x_t|x_{t-1}, x_0) \cdot q(x_{t-1}|x_0)}{q(x_t|x_0)}$$

由于三个分布 $q(x_t|x_{t-1})$ 、 $q(x_{t-1}|x_0)$ 和 $q(x_t|x_0)$ 均为高斯分布，利用高斯分布的乘除运算规则，可以证明 $q(x_{t-1}|x_t, x_0)$ 也是高斯分布：

$$q(x_{t-1}|x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I)$$

其中方差和均值分别为：

$$\tilde{\beta}_t = \frac{(1 - \bar{\alpha}_{t-1}) \cdot \beta_t}{1 - \bar{\alpha}_t}$$

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\bar{\alpha}_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t$$

后验依赖 x_0

注意 $q(x_{t-1}|x_t, x_0)$ 的均值 $\tilde{\mu}_t$ 同时依赖于 x_t 和 x_0 。然而在反向过程（生成过程）中，我们**没有** x_0 的信息——这正是需要神经网络来预测的原因。方差 $\tilde{\beta}_t$ 仅由噪声调度参数决定，不依赖 x_0 ，因此无需学习。

3.4 本章小结

ELBO 分解将训练目标转化为在每个时间步最小化反向转移分布与真实后验转移分布之间的 KL 散度。真实后验转移 $q(x_{t-1}|x_t, x_0)$ 是高斯分布，其方差可以直接计算，均值是 x_t 和 x_0 的线性组合。

4 反向过程的参数化

4.1 反向转移分布的建模

既然真实后验 $q(x_{t-1}|x_t, x_0)$ 是高斯分布，我们自然将反向转移 $p_\theta(x_{t-1}|x_t)$ 也建模为高斯分布：

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I)$$

其中：

- $\mu_\theta(x_t, t)$ ：由神经网络预测的均值，以 x_t 和时间步 t 为输入
- σ_t^2 ：方差，设定为 $\tilde{\beta}_t$ （与真实后验相同），**不需要学习**

为什么方差不需要学习？

由于我们要最小化 $D_{\text{KL}}(q(x_{t-1}|x_t, x_0) \| p_\theta(x_{t-1}|x_t))$ ，而两个高斯分布之间的 KL 散度在方差相同时简化为均值之差的 L2 范数。因此将 p_θ 的方差设为与 q 相同的 $\tilde{\beta}_t$ ，训练目标就只剩下匹配均值。后续的 Improved DDPM 论文探索了学习方差的方案，但基本 DDPM 中固定方差即可。

4.2 简化后的训练目标

将高斯形式代入 KL 散度，去噪匹配项 L_{t-1} 简化为：

$$L_{t-1} = \frac{1}{2\sigma_t^2} \|\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)\|^2 + C$$

其中 C 是与 θ 无关的常数。因此，训练目标就是让网络预测的均值 $\mu_\theta(x_t, t)$ 尽可能接近真实后验均值 $\tilde{\mu}_t(x_t, x_0)$ 。

训练的直觉

训练过程可以简洁地描述为：（1）采样一个干净数据 x_0 ；（2）采样时间步 t ；（3）通过前向跳跃分布采样含噪数据 x_t ；（4）用神经网络预测均值 $\mu_\theta(x_t, t)$ ；（5）计算预测均值与真实均值的 L2 损失并反向传播。

实际训练中，系数 $\frac{1}{2\sigma_t^2}$ 相当于一个与时间 t 相关的权重项。Ho et al. (2020) 发现，在实践中去掉这个权重（即对所有时间步使用均匀权重）效果也很好，甚至更好。这是 DDPM 论文中的一个重要经验发现。

4.3 本章小结

将反向转移建模为高斯分布并固定方差后，训练目标简化为均值的 L2 回归问题。权重项在实践中常被省略以简化训练。

5 三种等价的预测目标

由于真实后验均值 $\tilde{\mu}_t(x_t, x_0)$ 是 x_t 和 x_0 的线性函数，而 x_0 、 x_t 和噪声 ϵ 之间存在确定性关系，我们可以将均值预测重新参数化为三种等价的预测目标。

5.1 方式一：均值预测 (Mean Prediction)

最直接的方式——直接让神经网络预测反向转移的均值：

$$\mu_\theta(x_t, t) \approx \tilde{\mu}_t(x_t, x_0)$$

网络接受 x_t 和 t 作为输入，输出一个与 x_t 同维的均值向量。

训练损失为：

$$L_{\text{mean}} = \mathbb{E}_{t, x_0, x_t} \left[\|\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)\|^2 \right]$$

均值预测的特点

均值预测是最自然的参数化方式，它直接对应 ELBO 推导中的优化目标。然而，均值 $\tilde{\mu}_t$ 的尺度会随时间步 t 变化（因为它是 x_t 和 x_0 的加权组合，权重依赖 t ），这可能导致训练不够稳定。

5.2 方式二：干净样本预测 (Clean Sample Prediction / x_0 -Prediction)

由于 $\tilde{\mu}_t(x_t, x_0)$ 是 x_t 和 x_0 的线性函数，而 x_t 在推理时已知，我们可以让网络预测 x_0 而非直接预测均值：

$$\hat{x}_0 = f_\theta(x_t, t)$$

然后通过公式计算均值：

$$\mu_\theta(x_t, t) = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} f_\theta(x_t, t) + \frac{\sqrt{\bar{\alpha}_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t$$

等价的训练损失（忽略权重项）为：

$$L_{x_0} = \mathbb{E}_{t, x_0, \epsilon} \left[\|x_0 - f_\theta(x_t, t)\|^2 \right]$$

x_0 -预测的独特视角

x_0 -预测提供了一种引人入胜的采样直觉：在反向过程的每个中间步骤 t ，网络都会**预测最终输出 x_0 的期望值**，然后据此决定下一步 x_{t-1} 如何采样。这意味着扩散模型的采样过程是一个**迭代精化**（iterative refinement）过程——每一步都预测最终结果，然后逐步修正预测，而不是简单地直接输出。

x_0 预测不是一步生成

虽然每一步都预测了 x_0 ，但我们并不直接使用这个预测作为最终输出。相反，我们用 $\hat{x}_0$ 来计算反向转移的均值，然后采样 x_{t-1} ，接着在 x_{t-1} 上重新预测 $\hat{x}_0$ ，如此迭代直到 x_0 。每一步的 $\hat{x}_0$ 预测会逐渐变得更加精确。

5.3 方式三：噪声预测 (Noise Prediction / ϵ -Prediction)

这是 DDPM 论文中采用的参数化方式，也是实践中最常用的方式。

回忆前向跳跃分布的重参数化：

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

由此可以将 x_0 表示为 x_t 和 ϵ 的函数：

$$x_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon}{\sqrt{\bar{\alpha}_t}}$$

将其代入 $\tilde{\mu}_t$ 的表达式，可以得到以 x_t 和 ϵ 表示的均值：

$$\tilde{\mu}_t(x_t, \epsilon) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right)$$

因此，我们可以让网络预测噪声 ϵ 而非均值或 x_0 ：

$$\hat{\epsilon} = \epsilon_\theta(x_t, t)$$

然后通过公式计算均值：

$$\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right)$$

DDPM 的简化训练目标

Ho et al. (2020) 的核心发现：去掉权重项后，噪声预测的训练目标简化为极其简洁的形式：

$$L_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} \left[\|\epsilon - \epsilon_{\theta}(x_t, t)\|^2 \right]$$

其中 $t \sim \text{Uniform}(\{1, \dots, T\})$, $x_0 \sim q(x_0)$, $\epsilon \sim \mathcal{N}(0, I)$, $x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$ 。

这就是：给含噪图像，预测添加的噪声，最小化 L2 误差。

5.4 三种参数化的等价性与选择

三种参数化在数学上是完全等价的，因为它们之间可以互相转换：

- 从 $\hat{x}_0$ 可计算 $\hat{\epsilon} = \frac{x_t - \sqrt{\alpha_t} \hat{x}_0}{\sqrt{1 - \alpha_t}}$
- 从 $\hat{\epsilon}$ 可计算 $\hat{x}_0 = \frac{x_t - \sqrt{1 - \alpha_t} \hat{\epsilon}}{\sqrt{\alpha_t}}$
- 从 $\hat{x}_0$ 或 $\hat{\epsilon}$ 都可计算 $\hat{\mu}$

表 1: 三种预测目标的对比

参数化方式	预测目标	损失函数	备注
均值预测	$\tilde{\mu}_t$	$\ \tilde{\mu}_t - \mu_{\theta}\ ^2$	最自然但尺度依赖 t
x_0 预测	x_0	$\ x_0 - f_{\theta}(x_t, t)\ ^2$	迭代精化直觉
ϵ 预测	ϵ	$\ \epsilon - \epsilon_{\theta}(x_t, t)\ ^2$	DDPM 默认，最稳定

为什么噪声预测最常用？

实践中噪声预测 (ϵ -prediction) 是最广泛使用的参数化方式，主要原因在于**训练稳定性**：

1. **目标值的归一化**：噪声 $\epsilon \sim \mathcal{N}(0, I)$ 是标准正态分布的样本，天然具有零均值、单位方差的良好尺度特性。而 x_0 的尺度取决于数据集， $\tilde{\mu}_t$ 的尺度随 t 变化。
2. **梯度尺度一致**：由于噪声目标在所有时间步都具有相同的统计特性，梯度的尺度在不同时间步之间更加一致，这有利于训练稳定。
3. **无需额外归一化**：对于数据 x_0 ，通常需要归一化到特定范围；而噪声 ϵ 本身就是归一化的。

不同参数化适用于不同场景

虽然噪声预测在大多数场景中表现最好，但 x_0 预测在某些特定应用中更有优势，例如：图像编辑（需要直接估计干净图像）、视频生成（需要保持时序一致性）等。后续的 v-prediction 参数化则结合了两者的优点。选择参数化方式时需考虑具体任务需求。

5.5 本章小结

三种预测目标在数学上等价，但在实际训练中表现不同。噪声预测因其良好的归一化特性和训练稳定性成为默认选择。干净样本预测提供了直观的迭代精化视角。三种方式可以互相转换，实际实现中常在推理时利用这种转换关系。

6 DDPM 训练算法

6.1 完整训练流程

将前面的推导综合起来，DDPM 的训练算法可以写成极其简洁的形式：

1. 重复以下步骤直到收敛：

- (a) 从训练集中采样一个干净数据点： $x_0 \sim q(x_0)$
- (b) 均匀随机采样一个时间步： $t \sim \text{Uniform}(\{1, 2, \dots, T\})$
- (c) 采样一个标准高斯噪声： $\epsilon \sim \mathcal{N}(0, I)$
- (d) 计算含噪数据： $x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon$
- (e) 计算损失并更新参数： $\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(x_t, t)\|^2$

训练的简洁性

整个训练过程只需要几行 PyTorch 代码即可实现。没有对抗训练的不稳定性（如 GAN），没有变分推断的复杂性（如 VAE 的重参数化和 KL 项的平衡），只有一个简单的 L2 回归问题。这种简洁性是扩散模型被广泛采用的重要原因之一。

6.2 训练的伪代码实现

以下是使用 PyTorch 风格的简化实现：

```
1 # alpha_bar: precomputed cumulative product, shape [T]
2 # model: noise prediction network epsilon_theta(x_t, t)
3
4 for x0 in dataloader:
5     t = torch.randint(1, T+1, (batch_size,))
6     epsilon = torch.randn_like(x0)
7
8     # Forward jump: sample x_t directly from x_0
9     sqrt_alpha_bar = alpha_bar[t].sqrt()
10    sqrt_one_minus = (1 - alpha_bar[t]).sqrt()
11    x_t = sqrt_alpha_bar * x0 + sqrt_one_minus * epsilon
12
13    # Predict noise and compute loss
14    epsilon_pred = model(x_t, t)
15    loss = F.mse_loss(epsilon_pred, epsilon)
16
17    optimizer.zero_grad()
18    loss.backward()
19    optimizer.step()
```

$\bar{\alpha}_t$ 的预计算

在实现中, $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$ 应当在训练开始前**预计算**并存储为一个长度为 T 的向量。在每个训练步骤中, 根据随机采样的 t 索引对应的 $\bar{\alpha}_t$ 值即可。注意要正确处理 batch 维度上不同样本对应不同时间步的情况。

6.3 关于权重项的讨论

从 ELBO 严格推导, 去噪匹配项 L_{t-1} 前面有一个与 t 相关的系数 $\frac{1}{2\sigma_t^2}$ 。如果我们保留这个权重, 不同时间步的损失会被赋予不同的重要性。

Ho et al. (2020) 发现, 实践中**去掉权重** (对所有 t 均匀加权) 效果更好。直觉上, 这是因为:

- 当训练足够长时间且 t 被均匀采样时, 每个时间步都会被充分训练
- 均匀权重避免了某些时间步被过度强调而其他时间步被忽略的问题
- 实验上, 均匀权重的 FID 分数往往更好

权重项的理论与实践

虽然理论上权重项对应着正确的 ELBO 优化, 但去掉权重后的目标可以看作一种**重加权策略**, 实际上它更倾向于在高噪声水平上提高性能。后续的研究 (如 P2 weighting, Min-SNR weighting 等) 探索了各种自适应权重策略, 试图在理论正确性和实践效果之间找到更好的平衡。

6.4 本章小结

DDPM 的训练算法极为简洁: 采样数据、采样时间步、采样噪声、计算含噪数据、预测噪声、最小化 L2 损失。整个过程可以用不到 10 行 PyTorch 代码实现。权重项在实践中通常被省略。

7 DDPM 采样算法**7.1 反向过程采样**

训练完成后, 生成新样本的过程 (反向过程 / 采样过程) 如下:

1. 从标准高斯分布采样初始噪声: $x_T \sim \mathcal{N}(0, I)$
2. 对 $t = T, T-1, \dots, 1$:
 - (a) 用网络预测噪声: $\hat{\epsilon} = \epsilon_\theta(x_t, t)$
 - (b) 计算均值: $\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\alpha_t}} \hat{\epsilon} \right)$
 - (c) 若 $t > 1$, 采样 $z \sim \mathcal{N}(0, I)$; 若 $t = 1$, 令 $z = 0$
 - (d) $x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z$

3. 返回 x_0

采样的直觉：逐步去噪

采样过程可以理解为：从一团纯噪声开始，每一步都让网络「看一看」当前图像，预测其中有多少噪声，然后减去预测的噪声（同时加入少量新噪声以保持随机性），如此迭代直到获得干净图像。最后一步（ $t = 1$ ）不加噪声，因为我们要得到确定性的输出。

7.2 采样的伪代码

```
1 def ddpm_sample(model, T, alpha, alpha_bar, beta):
2     x = torch.randn(shape) # x_T ~ N(0, I)
3
4     for t in range(T, 0, -1):
5         epsilon_pred = model(x, t)
6
7         # Compute mean
8         mu = (1 / alpha[t].sqrt()) * (
9             x - (beta[t] / (1 - alpha_bar[t]).sqrt()) * epsilon_pred
10        )
11
12        if t > 1:
13            sigma = beta[t].sqrt() # or tilde_beta[t].sqrt()
14            z = torch.randn_like(x)
15            x = mu + sigma * z
16        else:
17            x = mu # No noise at final step
18
19    return x
```

Listing 2: DDPM 采样算法

采样速度是 DDPM 的主要瓶颈

DDPM 的采样需要迭代 T 步（通常 $T = 1000$ ），每一步都需要一次完整的神经网络前向传播。这意味着生成一张图像需要 1000 次前向传播，相比 GAN 的单次前向传播慢了约三个数量级。后续的 DDIM、DPM-Solver 等工作致力于减少采样步数，可在 10–50 步内获得接近的质量。

7.3 采样过程中的 $\hat{x}_0$ 可视化

当使用噪声预测网络时，在每个中间步骤，我们可以利用关系式计算当前对 x_0 的估计：

$$\hat{x}_0^{(t)} = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_{\theta}(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

在采样过程的早期（ t 接近 T ）， $\hat{x}_0^{(t)}$ 非常模糊，随着 t 减小，预测逐渐变得清晰。这种可视化直观地展示了扩散模型的“迭代精化”特性。

迭代精化与 Progressive Refinement

每一步的 $\hat{x}_0$ 预测可以理解为：在当前信息水平下，对最终结果的最佳猜测。随着去噪的推进，可用信息越来越多（噪声越来越少），预测自然越来越准确。这种 progressive refinement 的特性在实际应用中非常有用——例如可以在中间步骤引导生成方向（classifier guidance / classifier-free guidance）。

7.4 本章小结

DDPM 的采样过程是训练过程的“逆运算”：从纯噪声出发，逐步去噪直到获得干净图像。每一步预测噪声、计算均值、加入受控随机噪声。采样速度慢是主要局限，但后续有大量加速方法。

8 扩散模型为何能建模复杂分布

8.1 单步高斯 vs. 多步高斯

讲者在课堂上提出了一个深刻的问题：前向过程的转移是高斯的，反向过程的转移也被建模为高斯的，那么整个似然分布 $p_\theta(x_0|x_T)$ 是否也是高斯分布？如果是，那扩散模型岂不是和 VAE 一样受限？

答案是否。

非线性带来复杂性

虽然每一步反向转移 $p_\theta(x_{t-1}|x_t)$ 都是高斯分布，但整体似然分布 $p_\theta(x_0|x_T) = \int p_\theta(x_0|x_1)p_\theta(x_1|x_2)\cdots p_\theta(x_{T-1}|x_T) dx_{1:T-1}$ **不是**高斯分布。关键在于：每一步高斯的**均值**是由神经网络（非线性函数）预测的。

与前向过程不同——前向过程中均值是 x_{t-1} 的线性函数 ($\sqrt{1-\beta_t}x_{t-1}$)，因此前向过程的多步组合仍然是高斯的。而反向过程中均值是 x_t 的非线性函数（通过神经网络），这使得多步组合可以表达任意复杂的分布。

线性前向 vs. 非线性反向

这是理解扩散模型表达能力的关键：

- **前向过程**：均值是输入的**线性函数** → 多步组合仍为高斯 → 可推导闭式跳跃分布
- **反向过程**：均值是输入的**非线性函数**（神经网络） → 多步组合**不是**高斯 → 似然分布可以任意复杂

扩散模型巧妙地在“训练每一步简单”和“整体表达能力强”之间找到了平衡。

这也解释了为什么扩散模型需要多步：步数越多，非线性变换的层次越深，表达能力越强。这类似于深度神经网络比浅层网络更有表达力的道理。

8.2 本章小结

扩散模型的每一步转移虽然是高斯的，但由于反向过程使用非线性神经网络预测均值，多步组合的整体似然分布可以建模任意复杂的分布。这是扩散模型优于简单 VAE 的根本原因。

9 噪声预测与 Score Matching 的联系

9.1 从噪声预测到 Score Function

讲者在课程结尾预告了一个重要的联系：噪声预测网络 $\epsilon_\theta(x_t, t)$ 与 score-based 模型中的 score function 有着深刻的对应关系。

Score function 定义为对数概率密度对数据的梯度：

$$s_\theta(x_t, t) = \nabla_{x_t} \log p_t(x_t)$$

可以证明，噪声预测网络与 score function 之间存在如下关系：

$$\epsilon_\theta(x_t, t) \approx -\sqrt{1 - \bar{\alpha}_t} \nabla_{x_t} \log q_t(x_t)$$

DDPM 与 Score-Based 模型的统一

这一联系揭示了两个看似不同的生成模型框架——denoising diffusion probabilistic models (DDPM) 和 score-based generative models (SGM) ——本质上是等价的：

- DDPM 学习预测噪声 ϵ
- SGM 学习预测 score function $\nabla_x \log p(x)$
- 两者之间仅差一个与时间步相关的缩放因子

这种等价性由 Song et al. (2021) 在 Score SDE 框架中给出了统一的连续时间描述。

9.2 直觉理解

噪声预测可以被理解为回答这个问题：“ x_t 中有多少噪声？”而 score function 回答的是：“ x_t 应该往哪个方向移动才能增加其概率密度？”这两个问题在扩散过程的上下文中本质上是等价的——去除噪声的方向正是增加概率密度的方向。

9.3 本章小结

噪声预测与 score function 之间存在精确的数学对应关系，这建立了 DDPM 和 score-based 模型之间的桥梁，为后续的 Score SDE 统一框架奠定了基础。

10 总结与延伸

10.1 核心要点回顾

本讲详细推导了 DDPM 的训练和采样算法，核心内容可以总结为以下几点：

1. **前向过程**是预定义的高斯噪声注入过程，具有闭式的跳跃分布 $q(x_t|x_0)$ ，允许一步采样
2. **ELBO 分解**将训练目标转化为在每个时间步匹配真实后验转移分布
3. **真实后验** $q(x_{t-1}|x_t, x_0)$ 是高斯分布，方差可直接计算，均值依赖 x_0
4. **三种参数化**：均值预测、 x_0 预测、噪声预测在数学上等价，实践中**噪声预测**最稳定

5. **简化训练目标**: $L_{\text{simple}} = \mathbb{E} \|\epsilon - \epsilon_{\theta}(x_t, t)\|^2$, 优雅而高效
6. **表达能力**来自反向过程中神经网络引入的非线性: 每步高斯但整体非高斯
7. **噪声预测**与 score-based 模型有深刻联系, 为统一理论框架铺平道路

10.2 后续课程预告

讲者提到后续将讨论:

- 噪声预测与 score matching 的正式联系
- Score SDE 框架: 将离散时间扩散推广到连续时间
- 不同的前向过程定义方式
- 更快的采样方法 (DDIM 等)

10.3 拓展阅读

- **DDPM 原论文**: Ho, Jain, Abbeel. “Denoising Diffusion Probabilistic Models.” NeurIPS 2020.
- **Improved DDPM**: Nichol, Dhariwal. “Improved Denoising Diffusion Probabilistic Models.” ICML 2021. 探索了学习方差的方案。
- **Score SDE**: Song et al. “Score-Based Generative Modeling through Stochastic Differential Equations.” ICLR 2021. 统一了 DDPM 和 score-based 模型。
- **DDIM**: Song, Meng, Ermon. “Denoising Diffusion Implicit Models.” ICLR 2021. 提出了确定性采样方法, 大幅减少采样步数。
- **P2 Weighting**: Choi et al. “Perception Prioritized Training of Diffusion Models.” CVPR 2022. 探索了时间步权重的自适应策略。
- **v-prediction**: Salimans, Ho. “Progressive Distillation for Fast Sampling of Diffusion Models.” ICLR 2022. 提出了结合 x_0 预测和 ϵ 预测优点的 v-prediction 参数化。